David Mohaisen, Professor
University of Central Florida
Department of Computer Science
Cybersecurity and Privacy Cluster

https://cs.ucf.edu/~mohaisen/

Semantic Composition and Structured Meaning

① Introduction
② Language recursion
③ Parsing and mapping
④ Recursion vs RNNs
⑤ Scene parsing and TreeRNNs
⑥ Summary and Takeaways

Objectives

- Understand **semantic composition** and the principle of compositionality.
- Analyze challenges of **paraphrase, context sensitivity, and non-compositionality**.
- Compare **symbolic, distributional, and neural approaches** to meaning composition.
- Examine **recursion and hierarchical structure** in language.
- Understand **TreeRNN, TreeLSTM, and SU-RNN** for structure-aware modeling.

Computer Understanding of
NATURAL LANGUAGE
(CAP 6640)
Introduction

Semantic Interpretation of Language

How Can We Work Out the Meaning of Larger Phrases?

- Understanding meaning requires **semantic composition**, where the meaning of a whole is derived from the meanings of its parts and their interactions.
- Different **linguistic structures** lead to different interpretations:
  - Syntax and semantics work together to shape meaning.
  - Context and pragmatics influence interpretation.
- **Example Sentences:**
  - **"The snowboarder is leaping over a mogul."**
  - **"A person on a snowboard jumps into the air."**

→ These sentences express the same **core event** but differ in lexical/structure choice:

  - "Snowboarder" vs. "Person on a snowboard" (entity reference variation).
  - "Leaping over" vs. "Jumps into the air" (action variation).
  - Differences in **syntactic structure** but similar **semantic content**.

Semantic Composition: Meaning from Parts

- **Entities & Descriptive Terms**
  - "Snowboarder" → An agent participating in an action.
  - "Mogul" → A physical object, involved in spatial relations.
- **Predicates & Arguments**
  - "Leaping over" (motion verb with trajectory)
  - "Jumps into the air" (motion verb with a focus on vertical movement)
- **Paraphrase and Variability**
  - Different expressions can describe **the same underlying event**.
  - NLP models must **capture paraphrastic equivalence** through distributional, logical, or deep learning-based approaches.

Compositionality and Beyond

- **Traditional**: Meaning is built hierarchically (Meaning of whole = sum of parts). **Challenges**:
  - **Context sensitivity**: "jumps into the air" could imply different scenarios based on prior discourse.
  - **World knowledge**: Understanding a mogul is a snow obstacle type requires **external knowledge**.
  - **Non-compositionality**: Idioms and metaphorical expressions don't always follow strict rules.

- **Computational Approaches to Semantic Composition**
  - **Symbolic Approaches**: Lambda calculus, Combinatory Categorial Grammar (CCG), etc.
  - **Distributional Semantics**: Word vectors & sentence embeddings capture meaning similarity.
  - **Neural Models**: BERT/GPT use **contextualized embeddings** to represent meaning dynamically.

✅ Semantic composition is fundamental for understanding larger text units
✅ Computational NLP must handle both **systematic compositionality** and **context-dependent variations**.
✅ Hybrid models of **symbolic reasoning, neural embeddings, and external world knowledge** are the most promising.

Lang. Understanding and Compositionality in AI

- **Fundamental Principle**:
  - Human cognition and AI systems **construct meaning** by integrating smaller semantic units into larger structures.
  - **Compositionality**: The meaning of a whole is derived from the meanings of its components and their arrangement.

- **Why Is Compositionality Critical for Language Understanding?**
  - **Scalability**: Enables AI to process and understand **infinite linguistic expressions** from finite rules.
  - **Generalization**: Helps models handle **new combinations** of words and phrases.
  - **Interpretability**: Understanding how meaning emerges from parts supports more transparent AI models.

Lang. Understanding and Compositionality in AI

- **Example: Understanding Meaning Composition**
  - **Sentence 1:** ***"The snowboarder is leaping over a mogul."***
  - **Sentence 2:** ***"A person on a snowboard jumps into the air."***
- Both sentences convey **similar events**, despite differences in:
  - **Lexical choices**: "Snowboarder" vs. "Person on a snowboard"
  - **Action descriptions**: "Leaping over" vs. "Jumps into the air"
  - **Grammatical structure**
  - 👉 **Key Question**: How can AI recognize that these sentences describe the same event?
- **How AI Constructs Meaning from Smaller Units**
  - **Lexical Semantics:** Understanding individual word meanings via dictionaries (symbolic) or word embeddings (neural models).
  - **Syntactic Parsing**: Identifying grammatical structures (dependency trees, phrase structure trees).
  - **Semantic Role Labeling (SRL)**: Identifying the roles of entities (e.g., agent = snowboarder, action = jump, location = air).
  - **Contextualized Representation Learning**: Transformer-based models (BERT, GPT) encode meaning by capturing word relationships in **context**.

Computational Models for Compositionality

Approach	Description	Example
Symbolic	Logical rules & semantic grammars	Lambda calculus, CCG
Distributional	Vector-based similarity models	Word2Vec, GloVe
Neural Networks	Context-aware embeddings	Transformers (BERT, GPT)
Hybrid	Combines logic & deep learning	Neuro-symbolic AI

Language Recursion

Language Recursion

Cognitive Debate: Do Humans Actually Use Infinite Recursion?

- **Theoretical View**:
  - In formal linguistics, **recursion** is considered a fundamental property of language (Chomsky, 1957).
  - It allows **infinite embedding**: sentences can be expanded indefinitely by nesting structures inside each other.
- **Cognitive Reality**:
  - While recursion is **theoretically infinite**, human processing constraints suggest we **rarely** use deeply nested structures in natural speech.
  - Studies in psycholinguistics (e.g., Christiansen & Chater, 2015) indicate that excessive recursion **strains cognitive limits**.
  - Some languages (e.g., Pirahã, Everett 2005) have been argued to **lack recursion**, challenging the universality of this property.

Why Recursion is Natural for Language

- **Hierarchical Structure**: Language is **hierarchically organized**, making recursion a natural mechanism for generating complex expressions.

- **Embedded Clauses**: Sentences often contain other sentences or phrases:
  - **Relative Clauses**: *"The book [that I bought [which was recommended [by my professor]]]"*
  - **Center Embedding**: *"The person [standing next to [the man from [the company that purchased [the firm that you used to work at]]]]"*
    - **Noun phrase** → contains another **noun phrase** → contains another **noun phrase** (recursively).

Computational NLP and Recursion

- **Parsing Algorithms**:
  - Recursive structures create **challenges for syntactic parsers**.
  - Modern NLP models (e.g., **Transformers, RNNs**) handle recursion differently than **syntactic parsers** (e.g., CKY, Earley algorithms).

- **Challenges for AI**:
  - Neural models rely on **fixed-length representations**, making deeply recursive structures difficult to process.
  - Tree-based architectures (e.g., **Recursive Neural Networks**) attempt to explicitly capture **hierarchical structure**.

✅ Recursion is a natural mechanism for describing language structure.
✅ However, human cognition imposes practical limits on how deep recursion is used in natural language.
✅ AI systems struggle with deeply recursive structures, requiring hybrid approaches for robust parsing.

Computational NLP and Recursion

- The **Part-of-Speech (POS) parsing** for the sentences:
  - **Sentence:** *"He eats spaghetti with a spoon."*
  - **Sentence:** *"He eats spaghetti with meat."*

- **Key Observations**
  - **Ambiguity in Parsing:**
    - *"He eats spaghetti with a spoon."* → *"with a spoon"* modifies **eating** (how he eats).
    - *"He eats spaghetti with meat."* → *"with meat"* modifies **spaghetti** (what kind of spaghetti).
  - In **syntactic parsing**, the **attachment of "with"** can change the interpretation.

Constituency Parsing and Mapping

Constituency Parsing

- Constituency parsing represents the **syntactic structure** of a sentence by organizing words into **phrases** (e.g., noun phrases, verb phrases).
- Visualized as a **tree**, following the rules of a **context-free grammar (CFG)**.
  - **Sentence 1:** "*He eats spaghetti with a spoon*."
  - **Sentence 2:** "*He eats spaghetti with meat*."

```
S
├── NP (He)
├── VP
│   ├── V (eats)
│   ├── NP (spaghetti)
│   ├── PP
│   │   ├── P (with)
│   │   ├── NP
│   │   │   ├── Det (a)
│   │   │   ├── N (spoon)
```

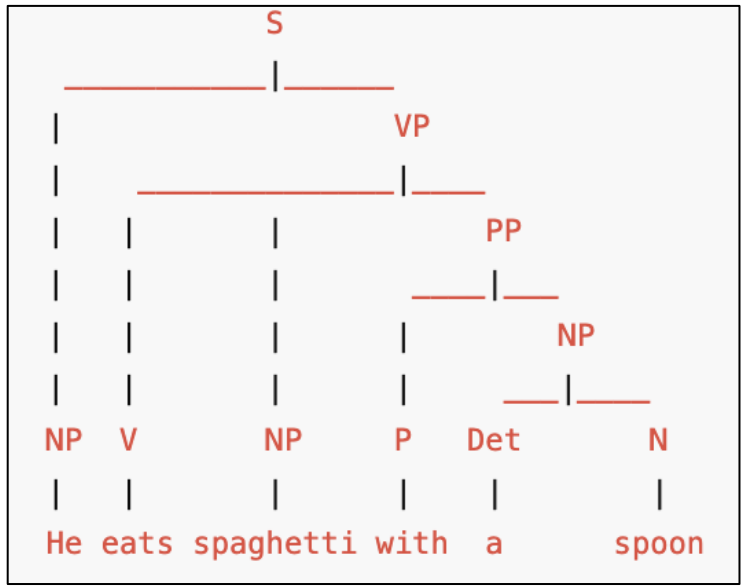
"with a spoon" attaches to **VP**, indicating the **manner of eating**.

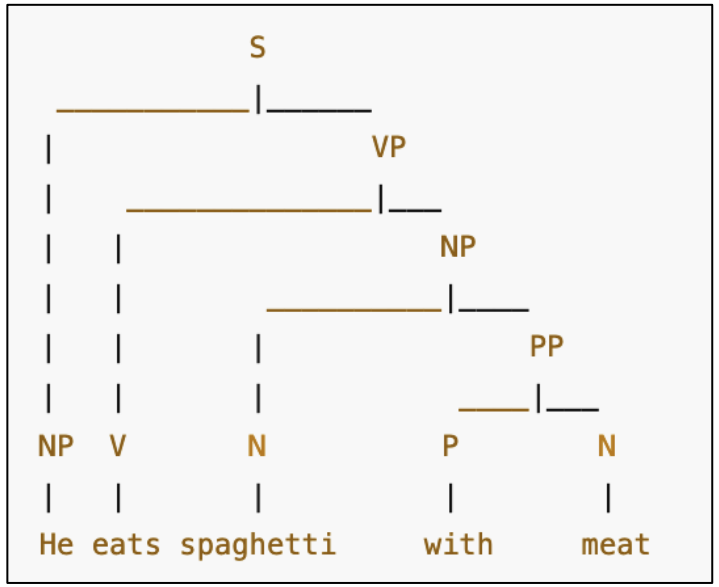
```
S
├── NP (He)
├── VP
│   ├── V (eats)
│   ├── NP
│   │   ├── N (spaghetti)
│   │   ├── PP
│   │   │   ├── P (with)
│   │   │   ├── N (meat)
```

"with meat" attaches to **NP (spaghetti)**, indicating **what kind of spaghetti**.

Constituency Parsing: Key Observation

- **Prepositional Phrase (PP) Attachment Ambiguity**
  - In **Sentence 1**, "with a spoon" modifies **the verb "eats"**, meaning "**How does he eat?**".
  - In **Sentence 2**, "with meat" modifies "**spaghetti**", meaning "**What kind of spaghetti?**".
    - This **PP attachment ambiguity** is a major challenge in NLP, affecting **parsing accuracy** and **semantic interpretation**.





Building on Word Vector Space Models

Recall Word Embedding

Ex 1: the country of my birth

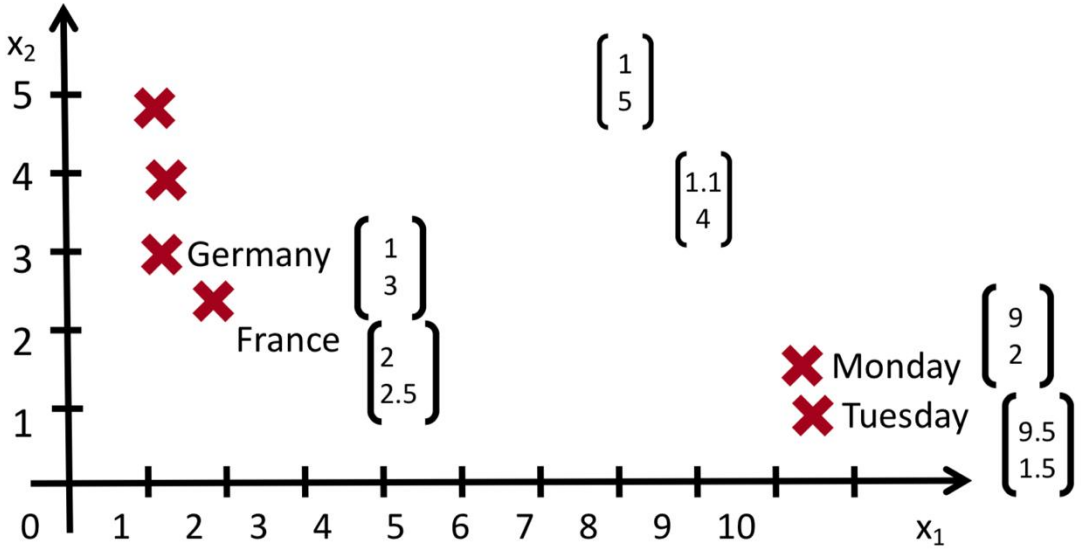
Ex 2: the place where I was born

Question

How can we represent the meaning of longer phrases?

Answer

By mapping them into the same vector space.



Mapping Phrases into Vector Space

1. **Word-Level Embeddings as Building Blocks:** Before constructing phrase representations, words must be represented as vectors. Common methods include:
   - **Static Embeddings**: Assign **fixed** vectors to words.
     - Examples: **Word2Vec, GloVe, fastText**
   - **Contextualized Embeddings**: Assign **dynamic** vectors based on context using **BERT, GPT, T5**
     - For example, the word "**bank**" will have different vector representations in:

     →*"He sat by the river **bank**."*

     →*"He deposited money in the **bank**."*

Mapping Phrases into Vector Space

2. Methods for Mapping Phrases into Vectors: Once word vectors are obtained, we can **compose** phrase representations. Several (5) approaches exist:

(A) Simple Averaging of Word Embeddings: Compute the vector representation of a phrase by **averaging** its word vectors.

- Example: Phrase: *"big apple"*
  - Word embeddings:
    - big → [0.2, 0.5, 0.1]
    - apple → [0.6, 0.8, 0.3]
  - Phrase embedding = **(0.2 + 0.6) / 2, (0.5 + 0.8) / 2, (0.1 + 0.3) / 2**
    - Result: [0.4, 0.65, 0.2]
- **Limitation**: Loses word order and syntactic structure.

Mapping Phrases into Vector Space

(B) Weighted Averaging (TF-IDF or Attention-based): Words that contribute more meaning (e.g., nouns, verbs) can be **weighted higher**.

- Example: *"He eats spaghetti with meat"* → "eats" and "spaghetti" contribute more than "with".
  - Weighting methods:
    - **TF-IDF**: Words with higher **importance in context** receive higher weights.
    - **Attention Mechanisms** (used in transformers like BERT) dynamically assign importance.

(C) Recurrent Neural Networks (RNNs, LSTMs, GRUs): Use **sequential models** to **encode phrases** into fixed-size vectors.

- Example: An **LSTM** processes "The snowboarder is leaping over a mogul" **word by word**, updating a hidden state.
  - **Advantage**: Captures **word order and long-range dependencies**.
  - **Limitation**: Struggles with **very long sentences** and suffers from **vanishing gradients**.

Mapping Phrases into Vector Space

(D) Transformer-based Models (BERT, GPT, T5): State-of-the-art method using **self-attention**.

- Example:
  - BERT provides **contextual phrase embeddings** by encoding full sentence meaning.
  - GPT predicts missing words and constructs **coherent representations**.
- **How it works**:
  - Sentences are tokenized and transformed into vectors.
  - Self-attention layers learn **phrase meaning from context**.
  - Output embeddings encode the entire phrase.

Mapping Phrases into Vector Space

(E) Sentence and Phrase Embeddings (Sentence-BERT, Universal Sentence Encoder): Specialized models trained to represent **entire phrases and sentences**.

- Examples:
  - **Sentence-BERT (SBERT)**: Fine-tuned BERT for sentence-level representations.
  - **Universal Sentence Encoder (USE)**: Trained on large corpora for **efficient sentence encoding**.
- **Key Benefit**: Phrases and longer text segments get more **meaningful representations**.

Mapping Phrases into Vector Space

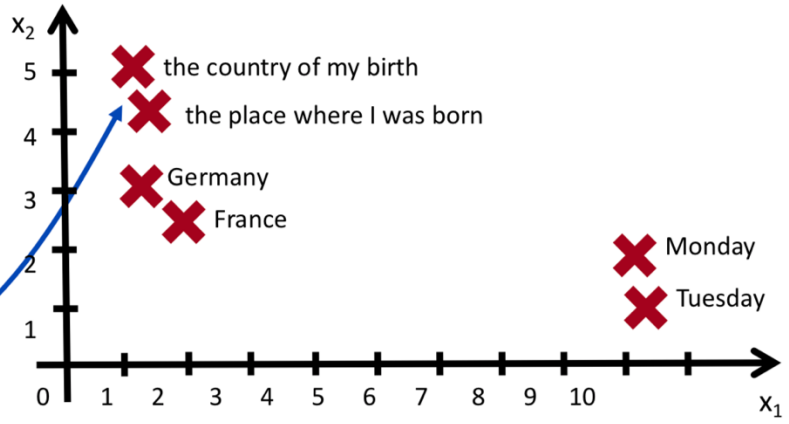
- **Visualization in Vector Space:** To understand **phrase embeddings**, we can use **dimensionality reduction** techniques like:
  - **t-SNE** (t-distributed Stochastic Neighbor Embedding)
  - **PCA** (Principal Component Analysis)
  - **UMAP** (Uniform Manifold Approximation and Projection)
- These methods **map high-dimensional vectors** into **2D or 3D space**, helping us visualize **semantic similarities** between phrases.

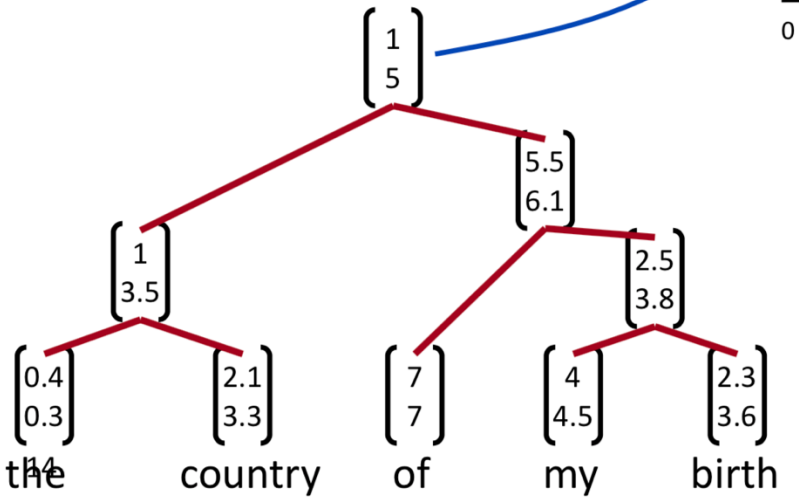
```python
from sentence_transformers import SentenceTransformer

# Load pre-trained Sentence-BERT model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Define phrases
phrases = ["He eats spaghetti with meat.", "The snowboarder is leaping over a mogul."]

# Compute embeddings
phrase_vectors = model.encode(phrases)

print(phrase_vectors.shape)  # Output: (2, 384) --> Each phrase is mapped to a 384-dimensional vector
```

Mapping Phrases into Vector Space

Socher, Manning, and Ng. ICML, 2011

Use principle of compositionality
The meaning (vector) of a sentence is determined by (1) the meanings of its words and (2) the rules that combine them.



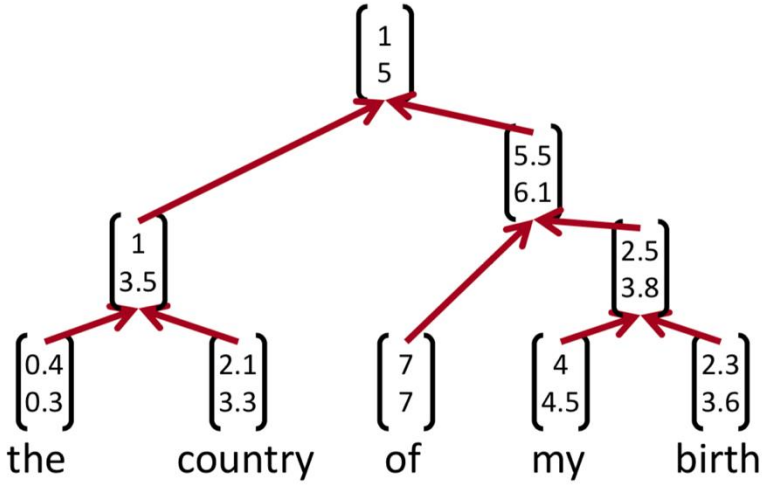


Models in this section can jointly learn parse trees and compositional vector representations

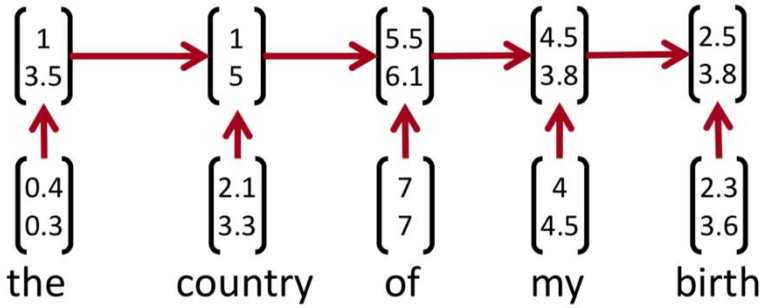
Recursive vs. Recurrent Neural Networks

Recursive vs. Recurrent Neural Networks

Recursive neural networks require a tree structure



Recurrent neural networks cannot capture phrases without prefix context and often capture too much of the last words in the final vector

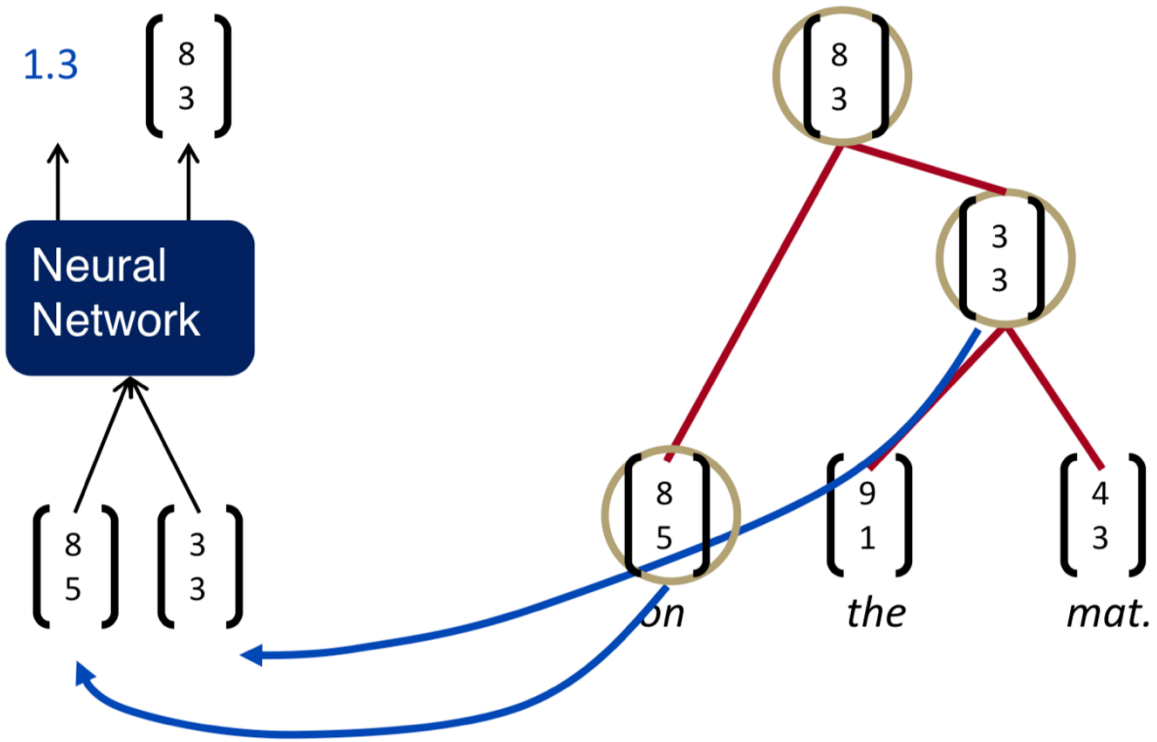


RNN for Structure Prediction

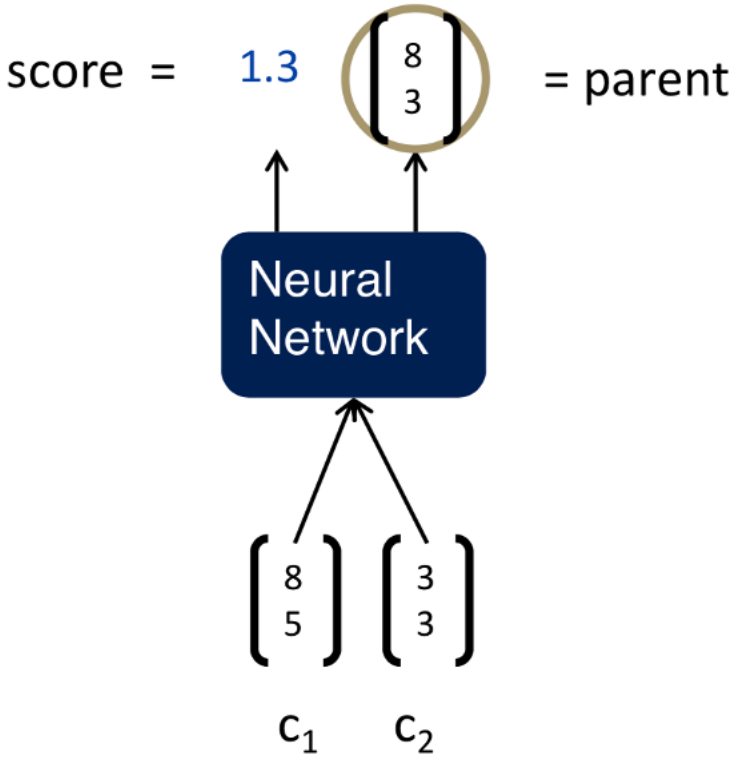
Input: two candidate children's representations

Output

1) The semantic representation if the two nodes are merged
2) Score of how plausible the new node would be



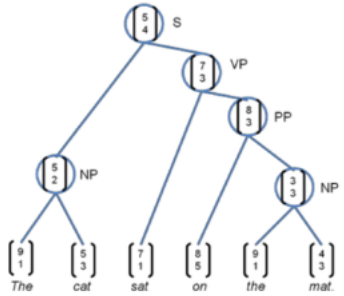
RNN Definition



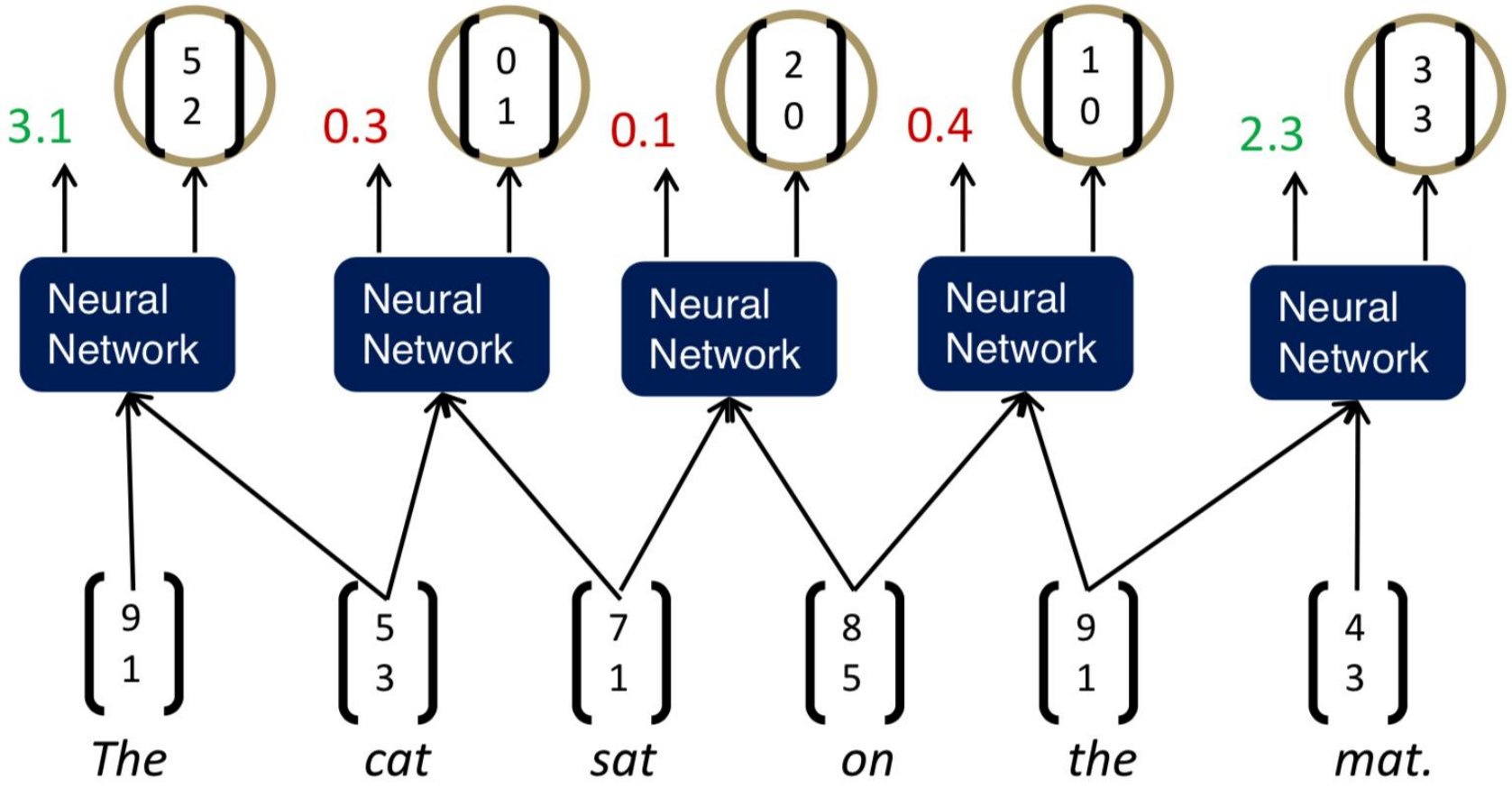
score $= U^T p$

$$p = \tanh\left(W\begin{bmatrix} c_1 \\ c_2 \end{bmatrix} + b\right),$$

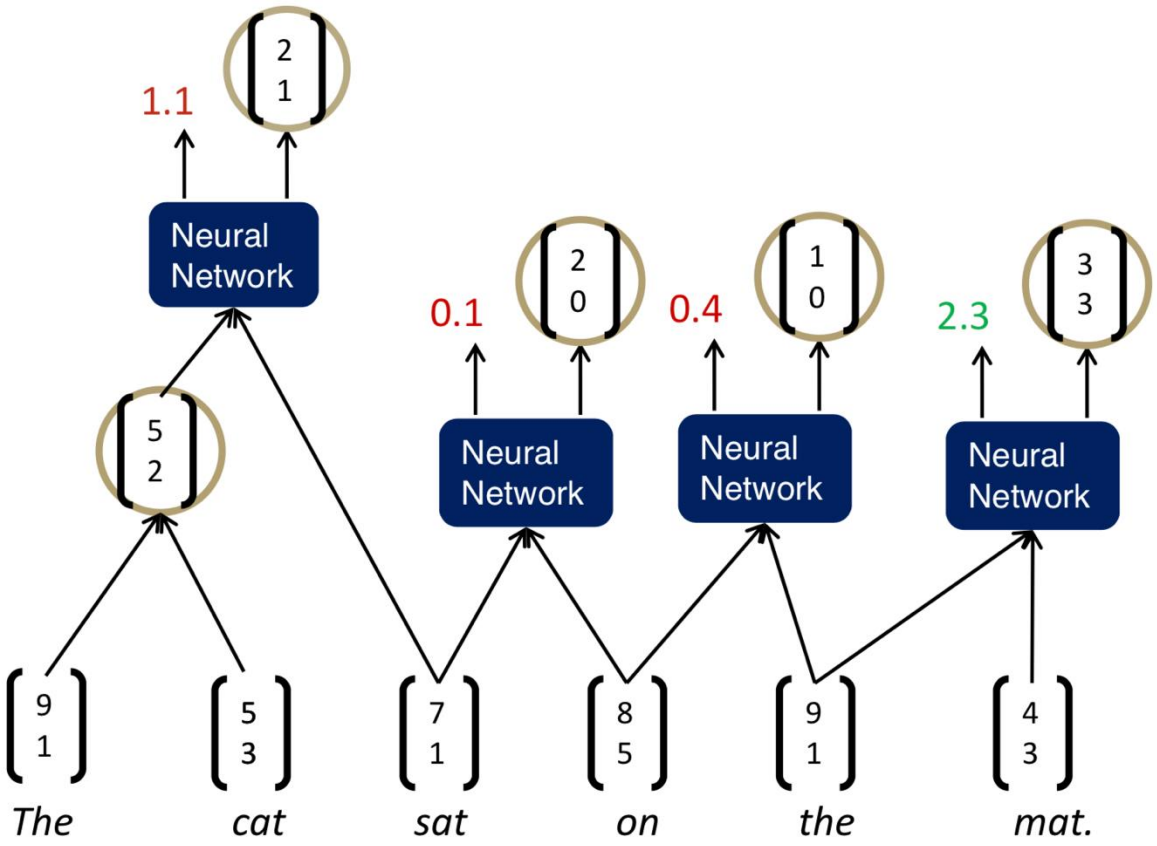
Same W parameters at all nodes of the tree



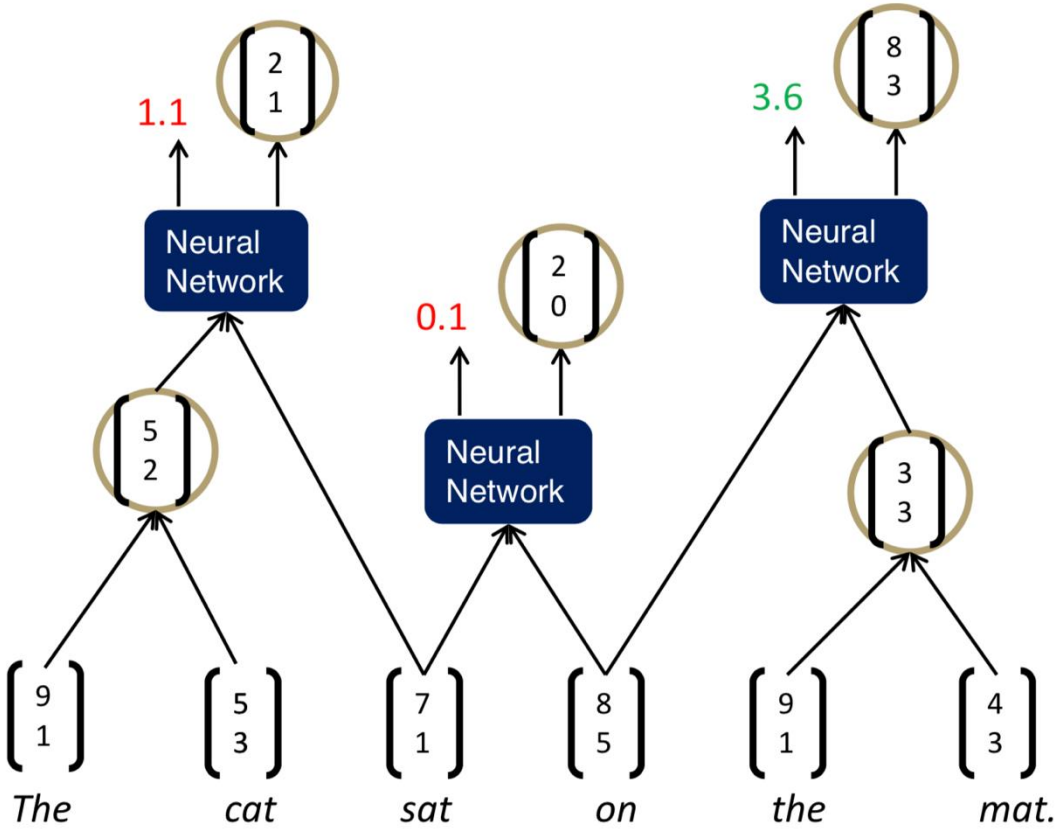
Parsing a Sentence with RNN



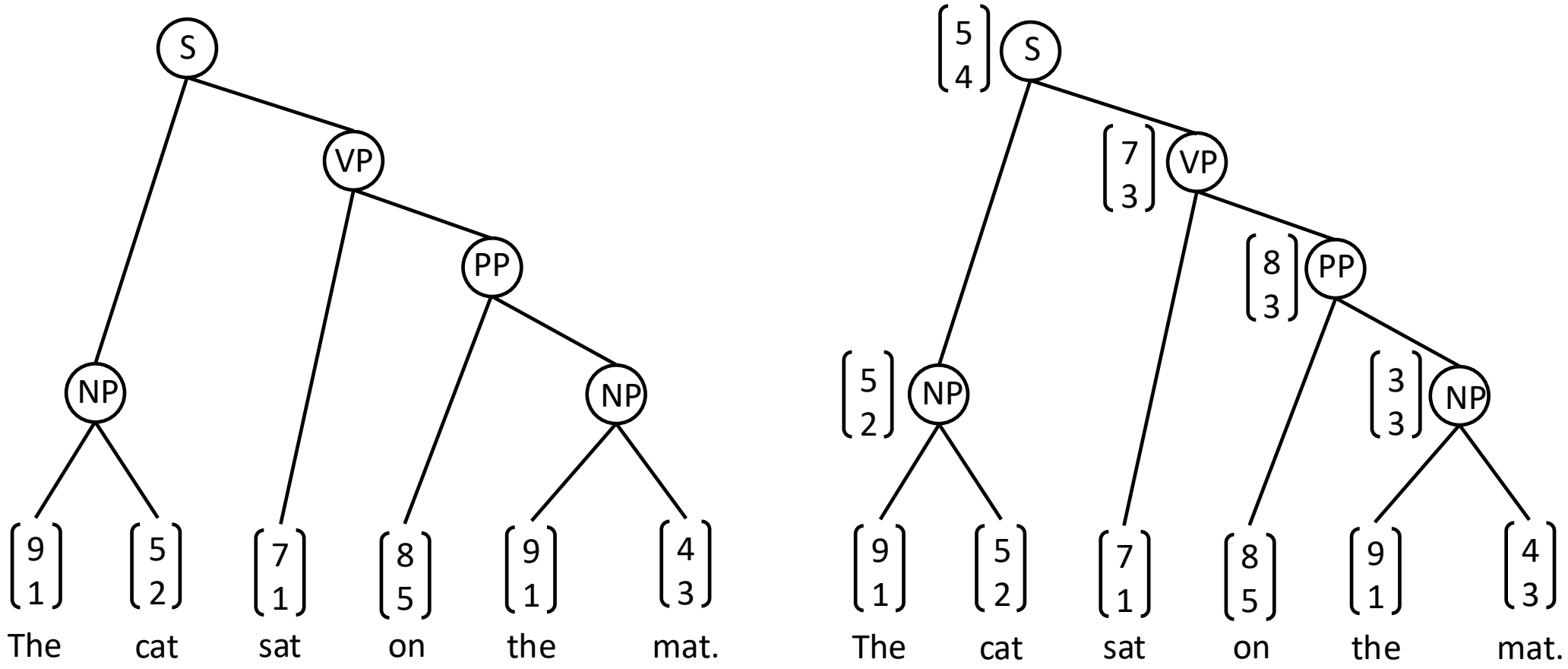
Parsing a Sentence with RNN



Parsing a Sentence with RNN



Constituency Sentence Parsing



Max Margin Framework

- The score of a tree is computed by the sum of the parsing decision scores at each node:

$$s(x, y) = \sum_{n \in nodes(y)} s_n$$

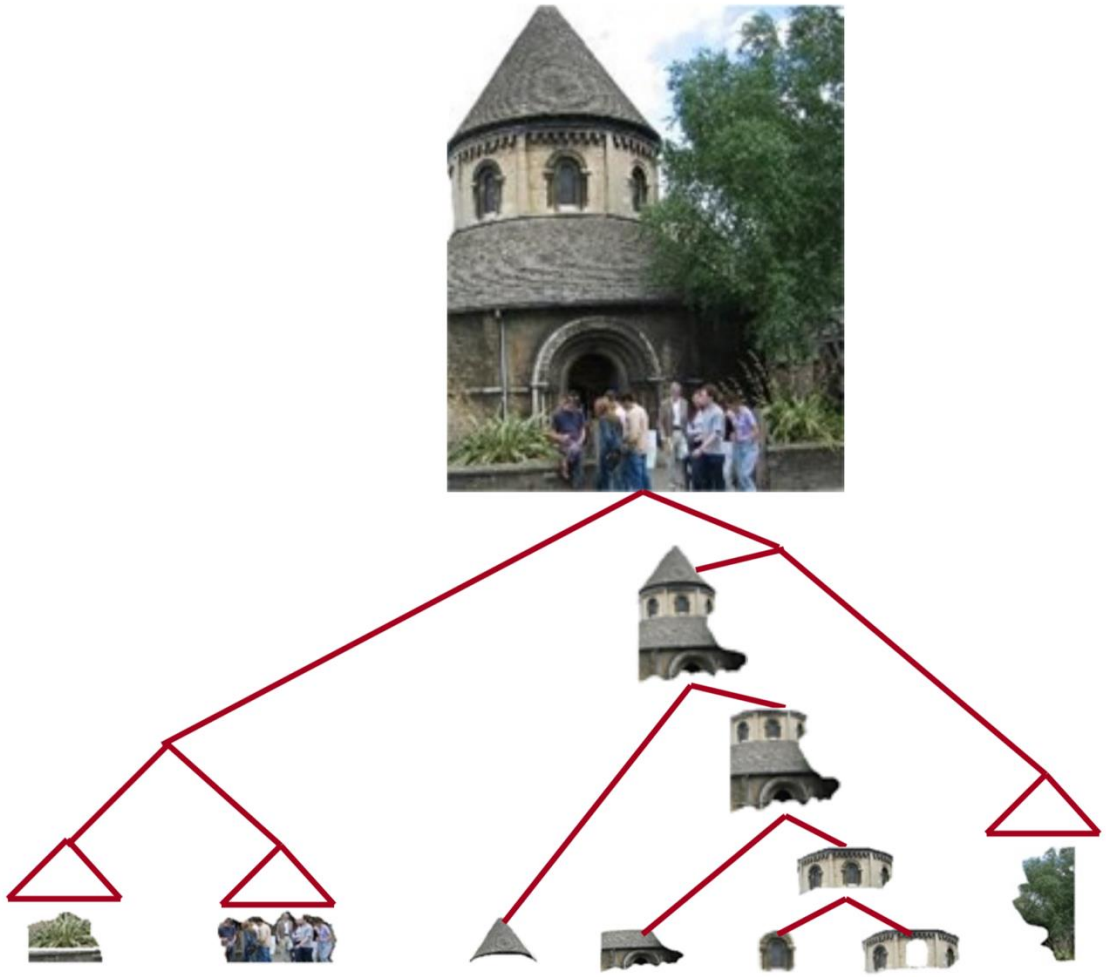
sentence

parse tree

Scene Parsing

Scene Parsing

- Scene parsing refers to **understanding and segmenting scene** into meaningful components. It is widely used in **computer vision** (image-based parsing) and **NLP** (understandin multi-sentence contexts).
- **Similar principle of compositionality**
  - The meaning of the scene image is also a function of smaller regions
    - How they combine as parts from the larger objective
    - And how the objects interact



Scene Parsing in NLP

- **Understanding the relationships between entities, actions, and objects in a narrative**.
  - **Event extraction** (who did what to whom, when, and where?)
  - **Semantic role labeling (SRL)** (assigning roles like agent, action, and object)
  - **Discourse parsing** (analyzing how sentences connect logically)
  - **Spatial and temporal reasoning** (where and when events occur)
- **Example Scene:** *"A snowboarder is leaping over a mogul. A person in the background is watching."* **Steps in Scene Parsing (NLP):**
  - **Syntactic Parsing:** Identify sentence structure.
  - **Named Entity Recognition (NER):** Extract key entities.
  - **Dependency Parsing:** Identify relationships between words.
  - **Event Extraction:** Recognize events and participants.
  - **Coreference Resolution:** Link pronouns to their referents.
  - **Spatial and Temporal Analysis:** Understand movement and event sequences.

Scene Parsing in NLP

- **Computational Models:**
  - **Semantic Role Labeling (SRL)** (e.g., AllenNLP, PropBank)
  - **Transformers (BERT, T5, GPT)** for context-aware event parsing
  - **Graph-based Models** (e.g., Scene Graphs) for structuring relationships

Scene Parsing in CV

- Scene parsing involves **segmenting an image into diff objects, surfaces, and bkgd**.
- **Key Techniques:**
  - **Semantic Segmentation:** Assigns each pixel to an object category.
  - **Instance Segmentation:** Differentiates multiple instances of the same object.
  - **Scene Graph Generation:** Creates a graph of objects and their relationships.
- **Example Scene Parsing in Images**
  - *"A snowboarder is leaping over a mogul while a person watches."*

Object	Action	Relation
Snowboarder	Leaping	Over mogul
Person	Watching	In background

Computational Models for Scene Parsing (Vision)

- Fully Convolutional Networks (FCN)
- DeepLab (ResNet, EfficientNet for segmentation)
- Transformers for Vision (ViT, Mask R-CNN)
- Graph Neural Networks (GNNs) for scene understanding

Scene Parsing in NLP+CV

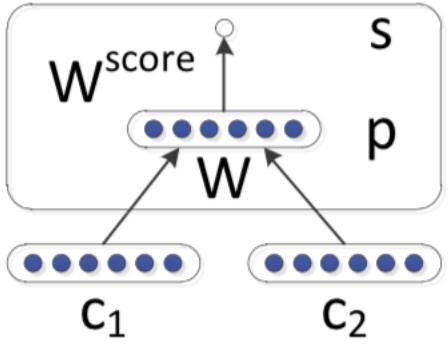
- **NLP + Vision: Multimodal Scene Parsing**
  - **Combining NLP & vision models** enhances understanding of **image-captioning, robotics, and autonomous systems.**
  - **Example:** A self-driving car needs to parse road scenes (**vision**) and traffic rules (**text**).
- **Multimodal Approaches**
  - **CLIP (OpenAI)**: Maps images and text into the same embedding space.
  - **LXMERT, ViLBERT**: Vision-Language Transformers for scene parsing.
  - **Neural-Symbolic Approaches**: Combine deep learning with logical reasoning.

✅ **Scene parsing in NLP** helps extract **events, actions, and relationships** from text.
✅ **Scene parsing in computer vision** enables **object recognition and segmentation**.
✅ **Multimodal approaches** (NLP + Vision) provide richer **contextual understanding**.

TreeRNNs

Simple TreeRNN

- **Tree Structure**: A simple **binary tree** is created with TreeNode.
- **Recursive Computation**:
  - If the node is a **leaf**, return its word embedding.
  - If the node has **children**, compute **representations recursively**.
- **Vector Composition**:
  - Child node embeddings are **concatenated**.
  - A **linear transformation** is applied.
  - **Tanh activation** ensures non-linearity.
- **Final Tree Representation**:
  - The **root node representation** encodes the entire sentence/tree



- **Applications**
  - **Sentence Representation** (Hierarchical meaning composition)
  - **Sentiment Analysis** (Sentences with structure-aware embeddings)
  - **Syntax-Aware Models** (Parsing-based NLP tasks)

Limitations of Single-Matrix TreeRNN

- While a **single weight matrix TreeRNN** can capture basic hierarchical structures, it **fails in complex composition** and **long sentence parsing**. Below are the key limitations:

(1) Lack of Interaction Between Words

- The TreeRNN **applies the same transformation** to all nodes, meaning it **does not model interactions between specific words**.
  - **Ex:** *"not bad"* → A simple TreeRNN may treat "not" and "bad" separately rather than capturing **negation**.
  - **Solution**: Use **gated mechanisms** like TreeLSTM, which selectively retains important information.

(2) Uniform Composition Function

- A single weight matrix **applies the same transformation to all nodes**, regardless of **syntactic category** (Noun Phrase, Verb Phrase), **function words** vs. **content words**, and **punctuation and structural elements**
  - **Ex:** *"The cat chased the mouse."* vs. *"The cat, which was hungry, chased the mouse."*A single-matrix TreeRNN does not differentiate **main clause vs. relative clause**.
  - Solution: TreeLSTM -- uses **gates to control information flow** at different tree nodes.

Limitations of Single-Matrix TreeRNN

(3) No Long-Range Dependencies: In deep hierarchical trees, **meaning gets diluted** due to **vanishing gradients**.

- **Example**: *"The athlete who won the race despite being injured was celebrated."* A simple TreeRNN struggles to connect "**athlete**" with "**celebrated**".
- Solution:
  - **TreeLSTM with Memory Cells**: Helps retain important information over long dependencies.
  - **Transformer-Based Parsing Models**: Like **BERT**, which processes entire sequences at once.

(4) Difficulty in Learning Complex Compositionality: Compositionality refers to how meaning is built from **smaller parts**.

- Example: *"red apple" vs. "red tape"* -- TreeRNN does **not differentiate composition of noun phrases vs. idiomatic expressions**.
- Solution: **Recursive Neural Tensor Networks (RNTNs)**: Capture higher-order interactions between words.

Limitations of Single-Matrix TreeRNN

(3) No Long-Range Dependencies: In deep hierarchical trees, **meaning gets diluted** due to **vanishing gradients**.

- **Example**: *"The athlete who won the race despite being injured was celebrated."* A simple TreeRNN struggles to connect "**athlete**" with "**celebrated**".
- Solution:
  - **TreeLSTM with Memory Cells**: Helps retain important information over long dependencies.
  - **Transformer-Based Parsing Models**: Like **BERT**, which processes entire sequences at once.

(4) Difficulty in Learning Complex Compositionality:
Compositi **er parts.**

- Example: **e**
  composit
- Solution: igher-order
  interactio

Issue	Solution
No real word interaction	Use TreeLSTM or Recursive Neural Tensor Networks (RNTNs)
Uniform composition function	Learn **different** weight matrices for different syntactic categories
Long-range dependencies lost	Use memory-enhanced models like TreeLSTM
Struggles with idioms	Use attention-based models (Transformers)

Syntactically-Untied RNN (SU-RNN)

- A **Syntactically-Untied RNN (SU-RNN)** is an improvement over standard **TreeRNNs**, addressing their **structural rigidity** and **uniform composition functions**.
- Unlike traditional **TreeRNNs**, which use a **single composition function** for all nodes, **SU-RNNs dynamically adjust transformation matrices based on syntactic roles**.
  - Different structures: NP, VP, and PP.

- Instead of a **single** weight matrix W, we use multiple matrices:
  - W_{NP} for Noun Phrases
  - W_{VP} for Verb Phrases
  - W_{PP} for Prepositional Phrases, etc.

Key Idea: Untying Weights by Syntactic Category

$$h_p = f(W_{syntactic_category} \cdot [h_{left}, h_{right}])$$

- h_p = Parent node representation
- $W_{syntactic_category}$ = Syntactic category-specific transformation
- h_{left}, h_{right} = Child node representations
- f = Activation function (e.g., Tanh, ReLU)

Syntactically-Untied RNN (SU-RNN)

- **Use Cases of SU-RNN**
  - **Sentence Representation** → Better hierarchical encoding of text.
  - **Semantic Role Labeling (SRL)** → Captures role-specific interactions.
  - **Syntax-Aware Sentiment Analysis** → Understands negation & dependencies.
  - **Machine Translation** → Maintains syntactic structure in translation.

Feature	TreeRNN	SU-RNN
Composition Function	Single matrix W	Category-specific matrices W_{NP}, W_{VP}
Word Interaction	Limited	Higher-order interactions captured
Parsing Accuracy	Lower	Better syntactic representation
Long-Range Dependencies	Weak	Improved representation

Takeaways

Takeaways

- Meaning emerges from **structured interaction of parts**, not just word sums.
- Pure compositionality is insufficient; **context and world knowledge** matter.
- Neural models approximate composition via **embeddings, attention, and hierarchy**.
- Recursive and tree-based models capture structure but face scalability limits.
- Hybrid and syntax-aware models better support **robust semantic understanding**.